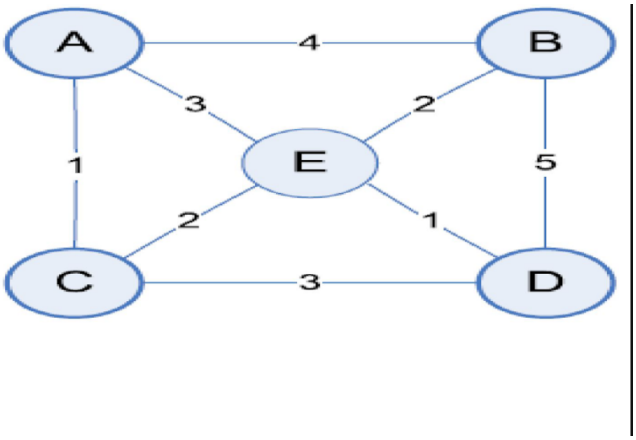
Name: V.Mahalakshmi
Reg No: 192521181
Subject: Artificial Intelligence
Subject Code: CSA1718

Travelling Salesperson Problem using DFS in Prolog

Aim

To write a Prolog program that uses Depth First Search (DFS) to find the shortest tour that starts from a city, visits all the cities exactly once and comes back to the starting city.

Given map



The map has 5 cities (A, B, C, D, E). The roads are two-way and the distances are:

A - B = 4
A - C = 1
A - E = 3
B - D = 5
B - E = 2
C - D = 3
C - E = 2
D - E = 1

There is no direct road between A and D, or between B and C.

Algorithm

1. Start at the starting city and mark it as visited.
2. From the current city, go to a neighbouring city that is not yet visited, add the road distance to the cost and go deeper (DFS).
3. When all cities have been visited, go back to the starting city if a road exists, and add that distance. This gives one complete tour.
4. If a city has no unvisited neighbour, or there is no road back to the start, backtrack and try the next choice.
5. Collect the cost of every tour found and pick the tour with the smallest cost.

Prolog searches depth first with backtracking, so the clauses below follow the DFS order automatically.

Program

```
% Travelling Salesperson Problem using Depth First Search

% road(City1, City2, Distance)   (roads are two-way)
road(a, b, 4).
road(a, c, 1).
road(a, e, 3).
road(b, d, 5).
road(b, e, 2).
road(c, d, 3).
road(c, e, 2).
road(d, e, 1).

connected(X, Y, D) :- road(X, Y, D).
connected(X, Y, D) :- road(Y, X, D).

cities([a, b, c, d, e]).

% tour(+Start, -Path, -Cost)
tour(Start, Path, Cost) :-
    cities(All),
    length(All, N),
    dfs(Start, Start, [Start], N, 0, RevPath, Cost),
    reverse(RevPath, Path).

% all cities visited: return to the start city
dfs(Current, Start, Visited, N, Cost0, [Start|Visited], Cost) :-
    length(Visited, N),
    connected(Current, Start, D),
    Cost is Cost0 + D.

% otherwise go deeper to an unvisited neighbour
dfs(Current, Start, Visited, N, Cost0, Path, Cost) :-
    length(Visited, L),
    L < N,
    connected(Current, Next, D),
    \+ member(Next, Visited),
    Cost1 is Cost0 + D,
    dfs(Next, Start, [Next|Visited], N, Cost1, Path, Cost).

% shortest_tour(+Start, -BestPath, -BestCost)
shortest_tour(Start, BestPath, BestCost) :-
    findall(Cost-Path, tour(Start, Path, Cost), Tours),
    keysort(Tours, [BestCost-BestPath|_]).

% print every tour found by DFS
show_all(Start) :-
    forall(tour(Start, Path, Cost),
           ( write(Path), write('  cost = '), write(Cost), nl )).
```

Queries

?- show_all(a).

```
[a,b,d,e,c,a]  cost = 13
[a,b,d,c,e,a]  cost = 17
[a,b,e,d,c,a]  cost = 11
[a,c,d,e,b,a]  cost = 11
[a,c,d,b,e,a]  cost = 14
[a,c,e,d,b,a]  cost = 13
[a,e,b,d,c,a]  cost = 14
[a,e,c,d,b,a]  cost = 17
```

?- shortest_tour(a, Path, Cost).

```
Path = [a, b, e, d, c, a],
Cost = 11.
```

Working

DFS finds 8 complete tours from A (the roads A-D and B-C do not exist, so other orders are dropped while backtracking). For the first tour, the search goes A, B, D, E, C and then returns to A: 4 + 5 + 1 + 2 + 1 = 13.

The cheapest tour costs 11. It is found twice, once in each direction:

A - B - E - D - C - A = 4 + 2 + 1 + 3 + 1 = 11

A - C - D - E - B - A = 1 + 3 + 1 + 2 + 4 = 11

keysort keeps the first of the equal tours, so the program reports A - B - E - D - C - A.

Result

Shortest tour: A - B - E - D - C - A

Minimum cost: 11